Part 4 – Personal Reflection

GymFlow: What the Build Revealed

Wiring up real-time Firestore data to the UI using Angular's RxJS took longer than I expected. The basic CRUD operations were straightforward, but combining the **members** and **plans** observable streams so the UI always reflected the latest data required careful thought — I used **combineLatest** to manage subscriptions properly, avoid memory leaks, and make sure the UI re-rendered cleanly whenever a membership was renewed or a new member added. Handling asynchronous state cleanly always takes longer than the happy path suggests.

If I were starting again, I would sketch the Firestore data model on paper before writing any code, since one structural change mid-way — splitting plans into their own collection — caused a cascade of small updates.

I feel least confident in Firestore data modelling at scale — specifically knowing when to denormalise for read performance versus keeping data referenced for write consistency. I chose to reference **plans** separately so pricing stays easy to update, but that means the frontend merges two collections client-side. I want to build stronger intuition for when duplicating data is the right call in NoSQL.